

软件架构设计					
项目编号	Drv_Ipcs_Core_Cmp	文件号	Drv_Ipcs_Core_Cmp_IPCS_DRV_SAS	文件版本号	1.0

Software Architecture Design

For IPCS Driver

IPCS Driver 软件架构设计

ROLES 角色	Name 姓名	Department 部门	Date 日期
AUTHOR(S) 作者:	倘亚朋	软件研发部	2026.4.15
REVIEWER(S) 审查:	安然/喻明睿/张帅/伊焕利/孔繁鑫/肖敏元/赵笏/宋晓婷/栗瑞江/WenKe	软件研发部/质量部/系统测试部	2026.4.16
APPROVER (S) 批准:	安然	软件研发部	2026.4.16

Document history: 变更历史

Version 版本	Date 日期	Editor 编辑人	Status 文档状态	Change description 变更简述
V0.1	2025.8.20	倘亚朋	Draft	Initial version for review 初版待评审
V0.8	2026.03.25	倘亚朋	已评审	更新缩略语部分 更新需求分配 更新外部接口 更新动态架构图
V1.0	2026.04.16	倘亚朋	待评审	增加架构约束与全局策略章节 增加全流程执行场景序列图 增加数据架构设计章节 更新架构总览，增加多 OS 场景描述 增加组件划分、组件关系章节 更新接口设计章节 更新组件间动态交互流程

CONTENTS 目录

1	INTRODUCTION 简介	4
1.1	CONFIDENTIALITY 保密性	4
1.2	PURPOSE OF THE DOCUMENT 文档目的	4
1.3	SCOPE 范围	4
1.4	REFERENCES 参考文件	5
1.5	ABBREVIATIONS 缩略语	5
2	ARCHITECTURAL CONSTRAINTS & POLICIES 架构约束与全局策略	6
2.1	TARGET 架构设计目标	6
2.2	CONSTRAINTS 架构设计约束	6
2.3	PRINCIPLES 架构设计原则	7
2.4	REQUIREMENTS ALLOCATION 需求分配	7
3	SOFTWARE ARCHITECTURE OVERVIEW 软件架构总览	10
3.1	MULTI-OS CONTEXT VIEW 多 OS 部署上下文图	10
3.2	STATIC STRUCTURE 静态结构图	10
3.3	DYNAMIC SCENARIOS 动态运行场景图	11
4	SOFTWARE ARCHITECTURAL DESIGN 软件架构设计	12
4.1	COMPONENT DESIGN 组件设计	12
4.1.1	<i>Principles</i> 组件设计原则	13
4.1.2	<i>Component Analysis</i> 组件分解分析	13
4.1.3	<i>Description Of Component</i> 组件描述	14
4.1.4	<i>Component Relationship view</i> 组件关系图	16
4.2	INTERFACE DESIGN 接口设计	18
4.2.1	外部接口 <i>IF_AppSvc</i> (端口 P1)	18
4.2.2	外部回调接口 <i>IF_AppSvc</i> (端口 P1)	23
4.2.3	内部接口 <i>IF_OSAbst</i> (端口 P4)	23
4.2.4	内部接口 <i>IF_HWAbst</i> (端口 P5)	24
4.3	FUNCTIONAL DESCRIPTION 功能描述	24
4.4	DATA ARCHITECTURE DESIGN 数据架构设计	25
4.4.1	<i>Instance Memory Layout</i> 通信实例级内存布局	25
4.4.2	<i>Managed Channel</i> 内存布局	26
4.4.3	<i>Unmanaged Channel</i> 内存布局	26
4.5	DYNAMIC ARCHITECTURE 动态架构图	27
4.5.1	<i>Use Case Sequence Diagram</i> 用例序列图	27

1 INTRODUCTION 简介

1.1 CONFIDENTIALITY 保密性

任何披露必须与负责的流程经理协调。

本文件过程说明仅限直接参与项目的人员查看。转让给其他方，尤其是 Star Gather 以外的合作伙伴，必须由项目负责人协调，并受开发合同中有关保密规定的约束。

1.2 PURPOSE OF THE DOCUMENT 文档目的

本文档用于给出 IPCS 驱动的统一化软件架构设计，作为 Linux 与 RTOS 两套实现的上层架构设计基线，用于支持设计评审、开发对齐、变更分析。

1.3 SCOPE 范围

本本文档适用于当前 IPCS 驱动工程的统一软件架构设计，覆盖以下两个部署形态：

- Linux 平台驱动实现
- RTOS 平台驱动实现

本文档关注的软件对象包括：

- 共享内存通信核心
- 队列与缓冲区管理机制
- OS 适配层
- HW 适配层
- 配置模型
- Linux UIO、cdev 与全内核模块部署适配层

本文档不覆盖：

- SoC 之外的系统级分区设计
- 安全分析、性能测试报告、验证报告
- 非驱动功能的软件业务逻辑

1.4 REFERENCES 参考文件

Reference ID 编号	Document Name 文档名称	Version 版本	Date 日期	Author 作者	Status 状态
1	Automotive SPICE® Process Assessment Model	2.5	2010-05-10	VDA	Release
2	MCAL 软件需求规范	1.0	2025-08-10	喻明睿	已发布
3	Drv_Ipcs_Core_Cmp MCAL 软件架构设计规范	1.0	2025-09-12	赵笃	已发布
4	Drv_Ipcs_Core_Cmp 软件架构规范	1.0	2025-6-1	薛艳江	已发布

1.5 ABBREVIATIONS 缩略语

Abbreviation 缩写	Meaning/Explanation 解释
API	Application Programming Interface
AUTOSAR	AUTomotive Open System ARchitecture
BD	Buffer Descriptor
BSW	Basic Software
CDD	Complex Device Driver
HAL	Hardware Abstraction Layer
IPCS	Inter-Processor Communication System
ISR	Interrupt Service Routine
MMU	Memory Management Unit
MPU	Memory Protection Unit
N/A	Not Applicable

OS	Operating System
OSAL	OS Abstraction Layer
RTOS	Real-Time Operating System
RTE	Runtime Environment
SHM	Shared Memory
SPSC	Single Producer Single Consumer

2 ARCHITECTURAL CONSTRAINTS & POLICIES 架构约束与全局策略

2.1 TARGET 架构设计目标

根据基线化的需求输入，IPCS 驱动的架构设计目标如下：

- 为同一芯片内部不同处理核心之间提供统一的共享内存通信机制
- 在 Linux 与 RTOS 两种运行环境下提供一致的外部通信能力与配置模型
- 支持多个通信实例与多个通信通道
- 支持由驱动管理缓冲区的通信模式，以及由应用管理通道内存的通信模式
- 隔离协议核心与 OS/HW 依赖，降低平台迁移与维护成本
- 支持中断驱动接收；在适用场景下支持 polling 接收
- 支持后续平台、OS 和集成方式的扩展

2.2 CONSTRAINTS 架构设计约束

本架构设计受到以下约束：

- Linux 与 RTOS 运行环境不同，但底层硬件通信模型一致
- 驱动必须基于共享内存与核间中断控制器工作
- 本地与对端的共享内存布局和 channel 配置必须对称
- 共享内存区域按底层平台约束使用，要求尽量对齐访问
- 资源受限环境下，架构需尽量保持核心逻辑简洁和可复用

2.3 PRINCIPLES 架构设计原则

为满足上述目标与约束，软件架构采用以下原则：

- 分层设计：协议核心、OS 适配、HW 适配、Linux 部署适配层彼此分离

- 核心复用：Linux 与 RTOS 共享同一套核心通信模型
- 配置驱动：实例、通道、缓冲区与中断关系由配置定义；配置结构化分层
- 接口稳定：上层使用统一驱动接口，不感知底层 OS/HW 差异
- 变体受控：平台差异和 OS 差异仅在明确的变体层内展开
- 一致性可校核：架构设计应能映射到现有实现并支持后续变更审查

2.4 REQUIREMENTS ALLOCATION 需求分配

本章落实 ASPICE SWE.2（软件架构设计）中对软件需求与架构元素之间可追溯分配（allocation）的要求：将已基线化的软件需求映射到本架构中的组件与工程属性，作为 SWE.1 → SWE.2 追溯的一环，并支撑 SWE.3（软件详细设计与单元实现）、SWE.4（软件集成与集成测试）及 SWE.5（软件合格性测试）的范围界定。

工作产品关系说明：

- 输入：架构设计的需求依据；
- 输出：下表为需求分配矩阵的架构视图；
- 多组件需求：若一条需求由多个组件协同满足，表中主责组件为实现与验证的首要归属；
- 组件代号：与本文档第 4 章一致（Drv_Ipcs_Core_Cmp ~ Drv_Ipcs_Linux_Adapt_Cmp）；

需求 ID	需求简述	主责组件 / 归属	协同 / 备注
IPCS_001	同核或异核上应用间点对点双向通信	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Queue_Cmp, Drv_Ipcs_Osal_Cmp, Drv_Ipcs_Hal_Cmp, Drv_Ipcs_Conf_Cmp	端到端路径：Drv_Ipcs_Conf_Cmp 配置对称布局；Drv_Ipcs_Core_Cmp 实例/通道；Drv_Ipcs_Queue_Cmp 队列与 BD；Drv_Ipcs_Osal_Cmp/Drv_Ipcs_Hal_Cmp 地址与通知
IPCS_002	部署于 AutoSAR OS 时满足 ASIL-D	过程类 + Drv_Ipcs_Osal_Cmp	安全目标分解、安全机制与证据在独立安全工作中产品化；架构上 Drv_Ipcs_Osal_Cmp 承担 AutoSAR 集成与 OS 交互边界
IPCS_003	内核与硬件平台、字节序无关	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Queue_Cmp, Drv_Ipcs_Hal_Cmp	可移植逻辑在 Drv_Ipcs_Core_Cmp/Drv_Ipcs_Queue_Cmp；平台相关集中在 Drv_Ipcs_Hal_Cmp；字节序通过可移植类型与显式转换策略在 Drv_Ipcs_Core_Cmp/Drv_Ipcs_Queue_Cmp 收敛
IPCS_005	作为复杂驱动在 AutoSAR OS 下运行	Drv_Ipcs_Osal_Cmp	与 AutoSAR CDD/OS 接口封装、调度与错误钩子对齐；Drv_Ipcs_Conf_Cmp 配置由集成方提供
IPCS_006	作为驱动在 ThreadX 下运行	Drv_Ipcs_Osal_Cmp	ThreadX 变体 OSAL：任务/中断/同步原语映射
IPCS_009	platform / os / phy / transport	过程类 + 全组件边界	目录与依赖方向约束架构分层；删除一层仅当上层不依赖该抽象时成立，需在集成配置中关闭对

	各层独立目录，可整体移除		应特性
IPCS_010	传输层设计支持基准与吞吐量估计	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Queue_Cmp, (传输层实现)	在传输路径保留可观测计数/时间戳或钩子，供基准测试构建使用；不与生产最小集强制绑定
IPCS_011	可在 Windows 主机上构建	过程类	构建系统、工具链与主机桩实现；不属 Drv_Ipcs_Core_Cmp ~ Drv_Ipcs_Linux_Adapt_Cmp 运行时职责
IPCS_012	主机单元测试、组件易于模型替换	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Queue_Cmp, Drv_Ipcs_Osal_Cmp, Drv_Ipcs_Hal_Cmp + 过程类	通过 OSAL/HWAL 接口注入替身；测试工程与 mock 属 SWE.3/SWE.5 支持
IPCS_013	信息安全与功能安全验证需模糊测试	过程类	SWE.5 /安全验证计划中的测试手段；架构上保持接口边界清晰以便 fuzz 目标收敛
IPCS_014	SHM 驱动支持多通信通道（数据流）	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Queue_Cmp, Drv_Ipcs_Conf_Cmp	多 channel 模型与共享内存布局
IPCS_015	通道/流内消息顺序保持	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Queue_Cmp	单通道队列语义与发送/接收顺序约定
IPCS_016	每通道支持多个缓冲池	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Queue_Cmp, Drv_Ipcs_Conf_Cmp	pool 配置与 Drv_Ipcs_Queue_Cmp 多队列管理
IPCS_017	全核多传输并行、高效核利用	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Osal_Cmp, Drv_Ipcs_Conf_Cmp	多 instance；Drv_Ipcs_Osal_Cmp 每核执行模型与中断亲和
IPCS_018	零拷贝 API 与实现	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Queue_Cmp	指针/BD 描述已有缓冲区，避免驱动内额外拷贝
IPCS_019	异步（基于通知）接收 API	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Osal_Cmp	回调由 Drv_Ipcs_Osal_Cmp 异步上下文触发；Drv_Ipcs_Core_Cmp 分发
IPCS_020	两通信端间不受干扰（内存保护）	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Conf_Cmp + 集成环境	共享内存分区与访问权限由布局（Drv_Ipcs_Conf_Cmp）与 SoC/OS 内存保护共同保证；Drv_Ipcs_Core_Cmp 遵守边界
IPCS_021	非托管通道，应用独占通道内存	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Conf_Cmp	unmanaged 模型与 ipcsShmUnmanaged* 路径
IPCS_022	虚拟中断不投递陈旧数据	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Queue_Cmp, Drv_Ipcs_Osal_Cmp, Drv_Ipcs_Hal_Cmp	序列号/队列状态与通知路径协调，避免在无效状态下回调
IPCS_023	丢失中断不导致数据丢失	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Queue_Cmp	依赖共享内存中的提交顺序与对端可见性；接收方可通过轮询或计数补偿（与 IPCS_039 协同）

IPCS_024	与 AUTOSAR 4.4 或更高版本兼容	Drv_Ipcs_Osal_Cmp, Drv_Ipcs_Conf_Cmp	接口与配置模型对齐 R4.4+ 集成约定
IPCS_025	池中缓冲区数量与大小可配置	Drv_Ipcs_Conf_Cmp	集成配置数据; Drv_Ipcs_Core_Cmp/Drv_Ipcs_Queue_Cmp 消费
IPCS_028	支持 核间中断作为收发通知	Drv_Ipcs_Hal_Cmp, Drv_Ipcs_Osal_Cmp	Drv_Ipcs_Hal_Cmp 平台实现; Drv_Ipcs_Osal_Cmp 挂接 Linux/RTOS 中断模型
IPCS_029	软件组件使用静态内存分配	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Queue_Cmp, Drv_Ipcs_Osal_Cmp	实现约束: 无堆或受限堆; 池与队列静态维度由 Drv_Ipcs_Conf_Cmp 定义
IPCS_030	设备端源码符合编码规范	过程类	编码标准与静态分析门禁
IPCS_031	多对内核 (多实例) 通信	Drv_Ipcs_Core_Cmp, Drv_Ipcs_Conf_Cmp	多 instance 与独立 SHM/IRQ 配置
IPCS_034	校验应用传入的公共 API 参数	Drv_Ipcs_Core_Cmp	各公共 API 入口参数与范围检查
IPCS_035	校验通道类型与所请求操作是否匹配	Drv_Ipcs_Core_Cmp	managed/unmanaged API 与 channel 类型一致性
IPCS_036	低延迟、小体积、可按需裁剪 HW/OS/传输	Drv_Ipcs_Core_Cmp ~ Drv_Ipcs_Linux_Adapt_Cmp, Drv_Ipcs_Conf_Cmp	可选编译与目录级特性开关; Linux 路径 Drv_Ipcs_Linux_Adapt_Cmp 按需链接
IPCS_038	按辰至汽车质量流程开发为生产级软件	过程类	项目管理、配置管理、问题解决与发布流程; 架构文档为 SWE.2 工作产品之一
IPCS_039	支持轮询方式 (IRQ_NONE)	Drv_Ipcs_Osal_Cmp, Drv_Ipcs_Hal_Cmp	Drv_Ipcs_Osal_Cmp ipcShmPollChannels / 轮询桥接; Drv_Ipcs_Hal_Cmp 在无硬件 IRQ 路径下的空操作或最小支持

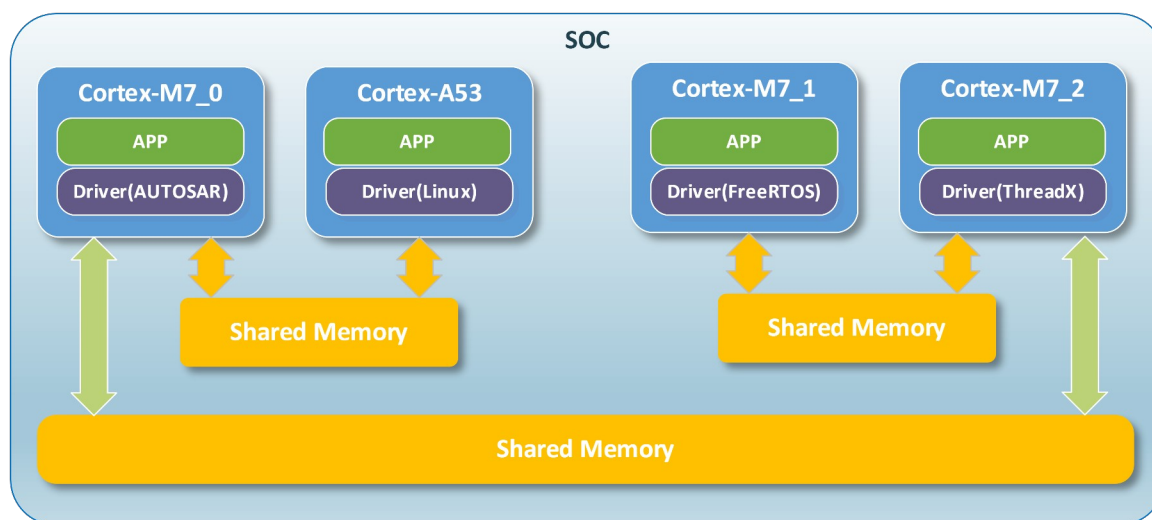
3 SOFTWARE ARCHITECTURE OVERVIEW 软件架构总览

3.1 MULTI-OS CONTEXT VIEW 多 OS 部署上下文图

IPCS 部署于多核 SoC, 为上层提供基于共享内存的核间数据传递。同一 SOC 之间不同核心可根据需求部署不同的 OS 适配层。IPCS Driver 位于上层 Application 和下层的 HAL 之间; 为上层提供同一的服务 API; 同时依赖:

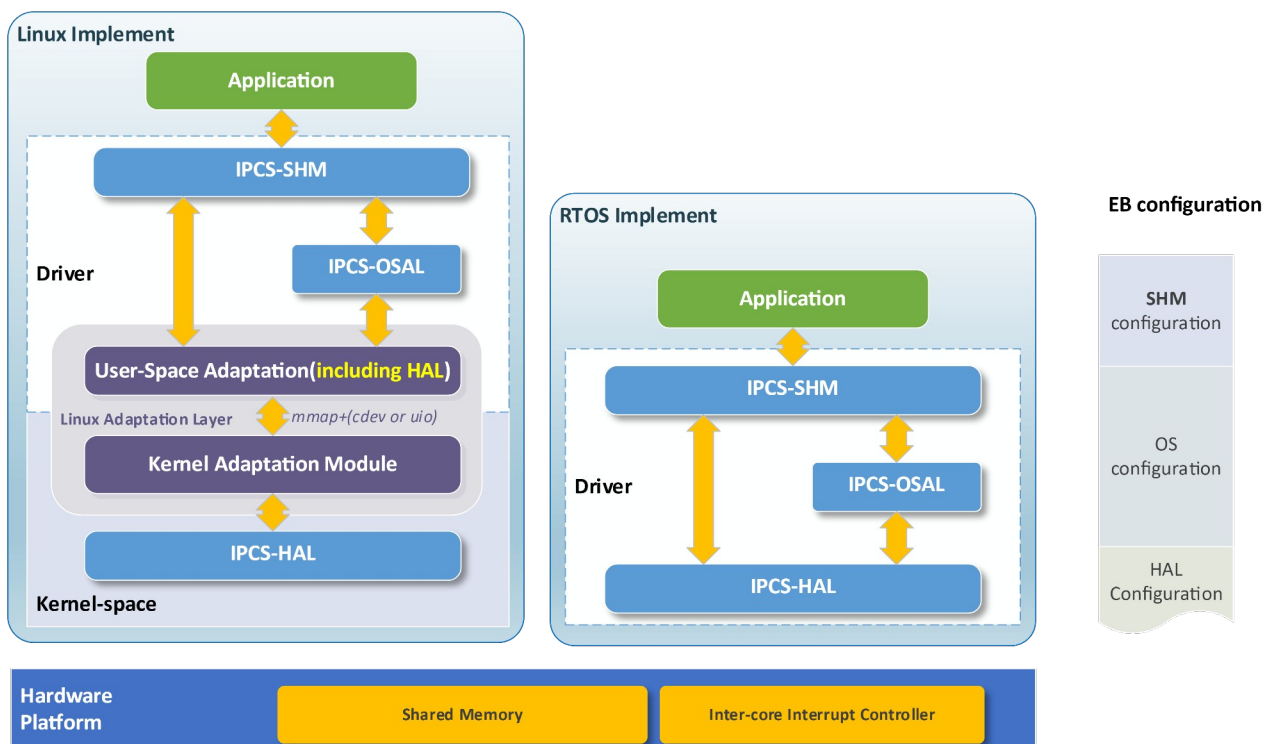
- 配置给定的本地/对端可访问共享内存窗口;

- 跨核通知 (HAL)
- OS 提供的映射、中断或调度能力 (OSAL)



3.2 STATIC STRUCTURE 静态结构图

IPCS Driver 软件分层架构如下图所示，其中，Linux Implement 是 IPCS-Linux 驱动静态结构图，RTOS Implement 是 RTOS-Linux 驱动静态结构图，两种不同 OS 部署结构采用相同的协议核心，提供相同的对外 API 服务接口，依赖相同的 OS 适配接口、HW 适配接口；特殊部分在于 Linux 平台根据部署需求加入了基于内存映射及 user-kernel 通信支持的适配层。

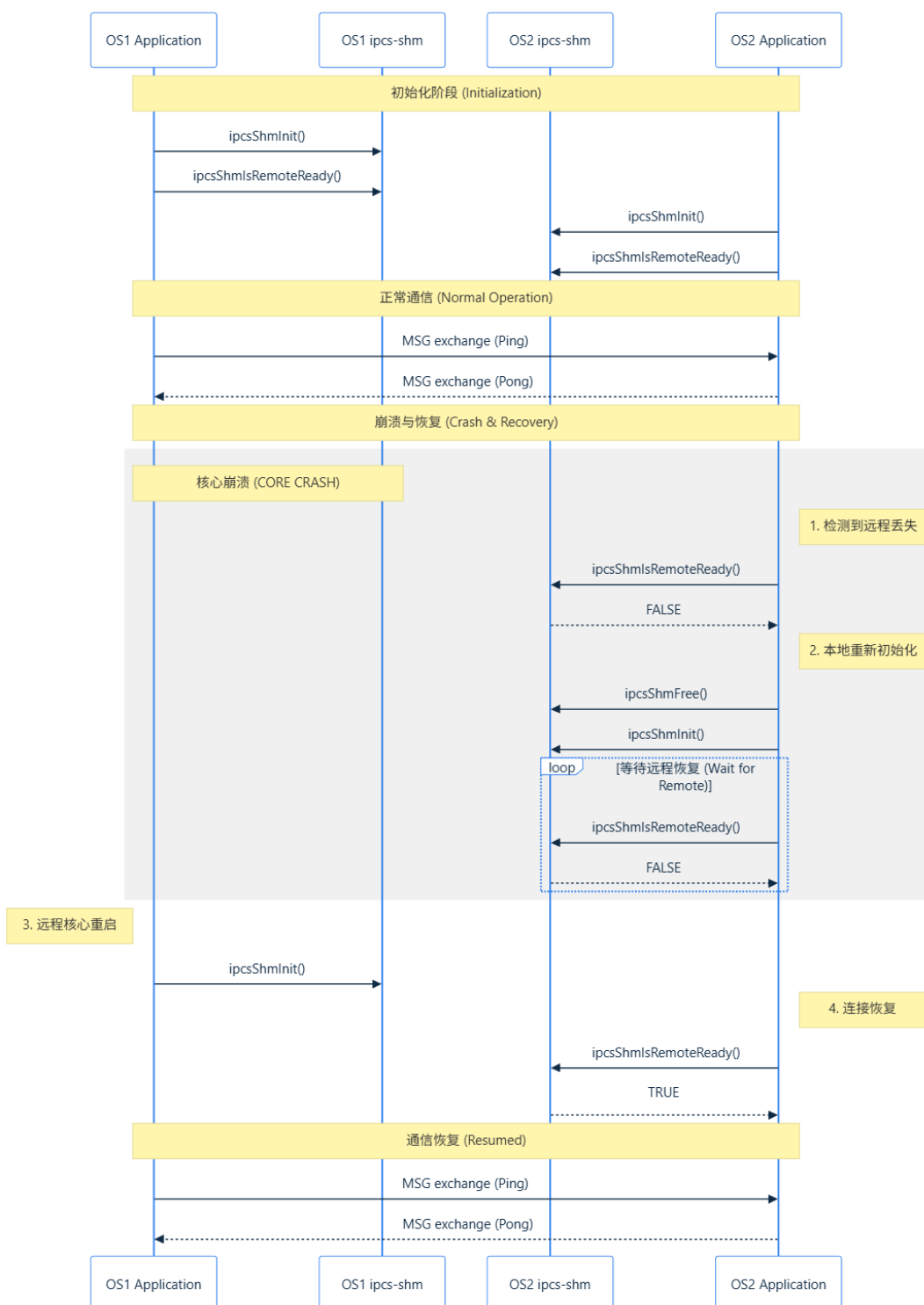


IPCS 驱动处于本地应用与底层核间通信资源之间，对上为应用或集成层提供统一通信接口，对下依赖共享内存和核间中断控制器，并与对端核上的对应通信实体配合完成数据交换，同时采用对应的分层化配置信息实现对 IPCS-SHM、OSAL、HAL 层级的配置。Application/Hardware Platform 不作为 Driver 的组成部分，用于表示驱动的上下文关系和职责边界；Kernel Adaptation Module 只作为 Linux 平台部署的适配层；如图中 Driver 部分标识，ICPS Driver 总体分为三个层级：

- **IPC-SHM:** 向上层 IPCS User 提供稳定服务入口与参数校验，完成 IPCS 的核心服务实现——实例/通道/队列、托管与非托管语义、接收调度；
- **IPC-OSAL:** 共享内存映射视图、中断或轮询注册、延迟处理上下文；对 IPCS-SHM 提供稳定的接口功能实现；对 IPCS-HAL 提供的固定接口调用，不直接操作硬件；依赖操作系统服务；
- **IPC-HAL:** 实现对底层硬件的抽象封装，为上层提供稳定的调用接口；职责是跨核通知使能/触发/清除及实例相关硬件状态；依赖底层硬件；
- **Kernel Adaptation Module:** 根据 IPCS-SHM CORE 运行与 kernel-space 或者 user-space 实现不同的配置，包括内存映射解决共享内存访问问题、uio 或者 cdev 解决 user-kernel 通信问题；

3.3 DYNAMIC SCENARIOS 动态运行场景图

IPCS Driver 核间共享内存通信的运行场景如下图所示，该时序图实现了基于对称配置的两个核心演示初始化、remote 就绪等待、共享内存通信、通信 crash、通信恢复的几个流程，图中接口符号参考 4.2 章节。



4 SOFTWARE ARCHITECTURAL DESIGN 软件架构设计

Component Design 组件设计

本章进一步说明在 IPCS 架构分层设计之下，软件被分解为哪些组件、各组件为何独立存在、组件之间如何保持高内聚低耦合，以及这些组件如何支撑后续详细设计、实现和测试。

4.1.1 PRINCIPLES 组件设计原则

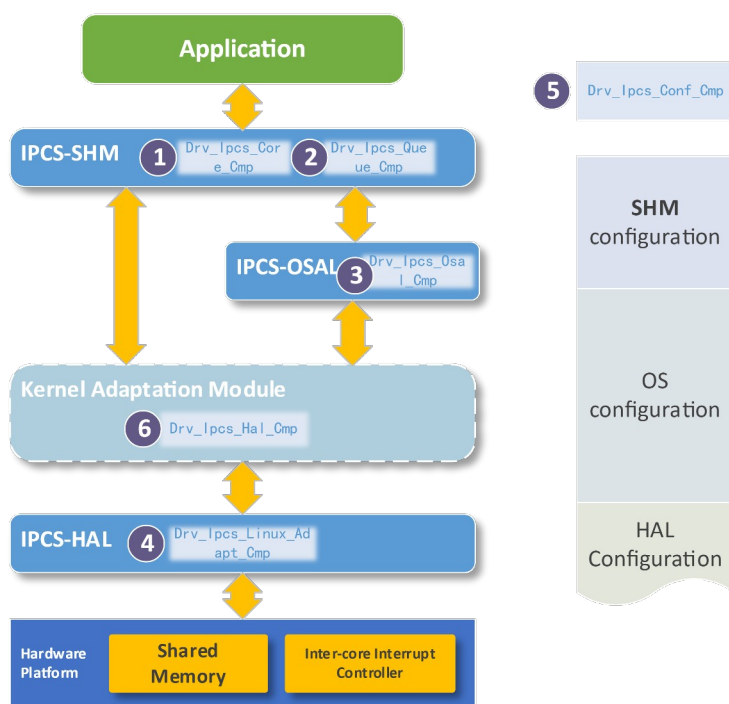
IPCS 组件设计遵循一下原则：

- 单一职责与高内聚：每个组件承担一类稳定职责，避免在同一组件中混合协议核心、OS 细节和平台寄存器访问逻辑。
- 变化隔离：将更可能随 OS、SoC 平台或 Linux 部署方式变化的内容隔离到独立组件中，降低对统一核心的影响范围。
- 接口清晰：组件之间仅通过明确的服务边界协作，避免跨层直接访问内部状态。
- 可验证与可替换：组件划分应支持主机侧单元测试、桩替换和平台变体验证，便于满足 IPCS_011、IPCS_012、IPCS_013 等需求。

4.1.2 COMPONENT ANALYSIS 组件分解分析

基于 4.1 组件设计原则及需求输入，组件设计分析如下所示，分层架构和组件的包含关系如下图所示。

- Drv_lpcs_Core_Cmp 通信核心组件 承担对外可见的驱动行为，是需求分配中的主要功能承载体。
- Drv_lpcs_Queue_Cmp 队列与缓冲管理组件 从 Drv_lpcs_Core_Cmp 中独立出来，用于保证共享内存队列算法、buffer 生命周期和描述符流转机制可复用、可验证。
- Drv_lpcs_Osal_Cmp OS 适配组件 独立承载中断模型、线程/任务、共享内存映射和 polling 桥接，避免 Drv_lpcs_Core_Cmp 直接依赖 Linux、AutosarOS、ThreadX 等 OS 机制。
- Drv_lpcs_Hal_Cmp HW 适配组件 独立承载平台相关核 ID、平台 IRQ 路由和寄存器访问，使平台差异不进入核心算法。
- Drv_lpcs_Conf_Cmp 配置组件 作为静态架构输入单独定义，确保实例、通道、pool、共享内存和 IRQ 拓扑不被散落到运行时逻辑中。
- Drv_lpcs_Linux_Adapt_Cmp Linux 部署适配组件 用于承载 Linux 特有的部署装配差异（全内核、UIO、cdev），防止这些装配逻辑污染统一的 OS/HW 抽象。



4.1.3 DESCRIPTION OF COMPONENT 组件描述

组件 ID	组件名称	主要职责
Drv_Ipcs_Core_Cmp	通信核心组件	提供 IPCS 驱动统一功能入口，负责实例管理、通道管理、共享内存布局、数据收发与接收分发
Drv_Ipcs_Queue_Cmp	队列与缓冲管理组件	提供基于共享内存的双环队列机制，支撑 BD 与 buffer 流转
Drv_Ipcs_Osal_Cmp	OS 适配组件	提供与操作系统相关的中断处理、共享内存地址管理与 polling 桥接
Drv_Ipcs_Hal_Cmp	HW 适配组件	提供与具体硬件平台相关的核间中断路由与通知能力
Drv_Ipcs_Conf_Cmp	配置组件	描述实例、通道、pool、共享内存区域、中断和核 ID 等配置数据
Drv_Ipcs_Linux_Adapt_Cmp	Linux 部署适配组件	提供 Linux 全内核模块、UIO 和 cdev 三种部署装配与用户态接入桥接

4.1.3.1 Drv_Ipcs_Core_Cmp 通信核心组件

Drv_Ipcs_Core_Cmp 是本架构的核心控制组件，其职责如下：

- 管理多个通信实例
- 管理每个实例下的多个通道
- 根据通道类型选择 managed 或 unmanaged 处理策略

- 管理 shared memory 中的通道布局
- 负责发送路径与接收路径的统一处理
- 调度接收预算，避免单个通道长期独占处理机会
- 对接应用侧回调
- 屏蔽 OS/HW 差异，向上提供统一接口

4.1.3.2 *Drv_Ipcs_Queue_Cmp 队列与缓冲管理组件*

Drv_Ipcs_Queue_Cmp 为 managed channel 提供通用的缓冲与描述符流转机制，其职责如下：

- 提供共享内存双环队列
- 支持空闲 buffer 的获取与归还
- 支持已发送数据的 BD 投递
- 保证队列机制可在 Linux 与 RTOS 之间复用

4.1.3.3 *Drv_Ipcs_Osal_Cmp OS 适配组件*

Drv_Ipcs_Osal_Cmp 负责把不同 OS 的执行与中断模型适配为统一驱动服务，其职责如下：

- 提供本地/远端 shared memory 地址服务
- 把中断事件转换为接收处理触发
- 在支持 polling 的场景下提供轮询桥接
- 负责 OS 相关资源的初始化与释放

4.1.3.4 *Drv_Ipcs_Hal_Cmp HW 适配组件*

Drv_Ipcs_Hal_Cmp 负责底层中断控制器与核间通知相关职责：

- 解释核 ID 与 IRQ 配置
- 执行接收中断使能与关闭
- 执行发送通知触发
- 执行接收中断状态清除
- 为不同平台提供统一的硬件服务抽象

4.1.3.5 *Drv_Ipcs_Conf_Cmp 配置组件*

Drv_Ipcs_Conf_Cmp 是架构中的静态设计输入，其职责如下：

- 描述 instance 数量与属性

- 描述 channel 数量、类型与参数
- 描述 pool 配置
- 描述共享内存区域
- 描述本地核、对端核和中断配置
- 描述回调入口

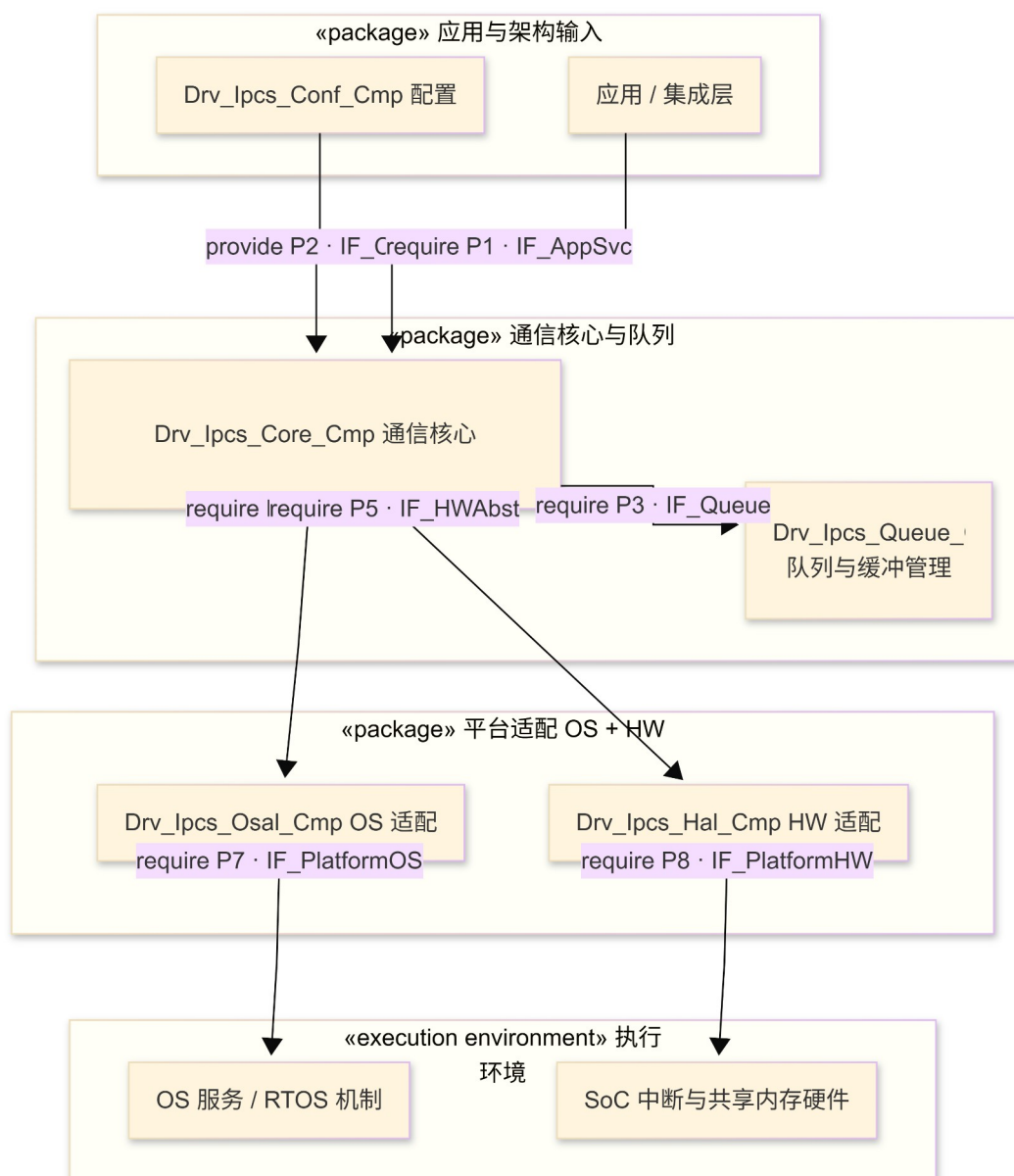
4.1.3.6 Drv_Ipcs_Linux_Adapt_Cmp Linux 部署适配组件

Drv_Ipcs_Linux_Adapt_Cmp 用于承载 Linux 部署形态下不属于统一通信核心算法、但决定核心位于内核态还是用户态的装配与桥接逻辑。其职责如下：

- 组织 Linux 全内核模块、UIO 和 cdev 三种部署形态
- 为用户态接入提供内核薄桥接能力
- 对接 Linux UIO 框架和字符设备机制
- 在不影响统一核心设计的前提下隔离 Linux 特定部署差异

4.1.4 COMPONENT RELATIONSHIP VIEW 组件关系图

组件关系图如下所示，它展示组件的依赖关系和职责边界；下表根据依赖关系定义了软件组件、软件-Hardware、外部依赖（OS）之间的端口符号化约定。



端 口	UML 接口	提供方	需要方	说明
P 1	IF_AppSvc	Drv_Ipcs_Core_Cmp	应用/集成层	初始化、发送、接收、状态管理等对上能力
P 2	IF_CfgIn	Drv_Ipcs_Conf_Cmp	Drv_Ipcs_Core_Cmp	实例/通道/pool/共享内存/中断/回调等静态配置
P 3	IF_Queue	Drv_Ipcs_Queue_Cmp	Drv_Ipcs_Core_Cmp	共享内存队列与缓冲/BD 流转
P 4	IF_OSAbst	Drv_Ipcs_Osal_Cmp	Drv_Ipcs_Core_Cmp	地址映射、IRQ 桥接、轮询或调度等 OS 抽象
P 5	IF_HWAbst	Drv_Ipcs_Hal_Cmp	Drv_Ipcs_Core_Cmp	通知、清除、使能等平台 HW 抽象
P	IF_PlatformOS	OS 执行环境	Drv_Ipcs_Osal_Cmp	任务、中断、映射、同步等底层

7			p	OS 能力
P8	IF_PlatformHW	HW 执行环境	Drv_Ipcs_Hal_Cmp	核间中断、寄存器、共享内存物理资源

4.2 INTERFACE DESIGN 接口设计

4.2.1 外部接口 IF_APPSVC (端口 P1)

根据架构设计及组件设计，IPCS 通过 IPCS-SHM 核心组件提供对上的统一公共接口，用于满足初始化、发送、接收和状态管理需求。

4.2.1.1 ipcsShmInit(P1-OP01)

Field	Content	
Service Name	ipcsShmInit	
Syntax	int8_t ipcsShmInit(const struct IpcsShmInstancesCfg *cfg)	
Service ID [hex]	N/A	
Sync/Async	Synchronous	
Reentrancy	Non Reentrant	
Parameters (in)	cfg	The pointer of multiple instances
Parameters (inout)	None	None
Parameters (out)	None	None
Return value	int8_t	Error code
Description	Initialize all instances and resources	
Available via	ipcs-shm.h	

4.2.1.2 ipcsShmFree(P1-OP02)

Field	Content	
Service Name	ipcsShmFree	
Syntax	void ipcsShmFree(void)	
Service ID [hex]	N/A	
Sync/Async	Synchronous	

Reentrancy	Non Reentrant	
Parameters (in)	None	None
Parameters (inout)	None	None
Parameters (out)	None	None
Return value	None	None
Description	Release all instances and resources	
Available via	ipcs-shm.h	

4.2.1.3 ipcsShmAcquireBuf(P1-OP03)

Field	Content	
Service Name	ipcsShmAcquireBuf	
Syntax	void *ipcsShmAcquireBuf(const uint8_t instance, uint8_t chan_id, uint32_t mem_size)	
Service ID [hex]	N/A	
Sync/Async	Synchronous	
Reentrancy	Conditionally Reentrant Reentrancy is not supported for the same channel, but is supported for different channels/different instances	
Parameters (in)	Instance	The index of instance
	chan_id	The identify of specified channel
	mem_size	Allocated memory size
Parameters (inout)	None	None
Parameters (out)	None	None
Return value	void *	The buffer pointer or NULL
Description	Request/allocate a writable send buffer for the managed channel; this buffer must not be used until the peer side is ready.	
Available via	ipcs-shm.h	

4.2.1.4 ipcsShmReleaseBuf(P1-OP04)

Field	Content	
Service Name	ipcsShmReleaseBuf	

Syntax	int8_t ipcsShmReleaseBuf(const uint8_t instance, uint8_t chan_id, const void *buf)	
Service ID [hex]	N/A	
Sync/Async	Synchronous	
Reentrancy	Conditionally Reentrant Reentrancy is not supported for the same channel, but is supported for different channels/different instances	
Parameters (in)	Instance	The index of instance
	chan_id	The identify of specified channel
	buf	Buffer pointer to release
Parameters (inout)	None	None
Parameters (out)	None	None
Return value	int8_t	Error code
Description	Once the receiver has completed processing, free the managed channel buffer, return it to the buffer pool, and transfer it back to the peer's transmit path through BD semantics.	
Available via	ipcs-shm.h	

4.2.1.5 ipcsShmTx(P1-OP05)

Field	Content	
Service Name	ipcsShmTx	
Syntax	int8_t ipcsShmTx(const uint8_t instance, int chan_id, void *buf, uint32_t size)	
Service ID [hex]	N/A	
Sync/Async	Synchronous	
Reentrancy	Conditionally Reentrant Reentrancy is not supported for the same channel, but is supported for different channels/different instances	
Parameters (in)	Instance	The index of instance
	chan_id	The identify of specified channel
	buf	Buffer pointer to payload
	size	Payload size
Parameters (inout)	None	None

Parameters (out)	None	None
Return value	int8_t	Error code
Description	Submit the managed channel transmission, push the BD, and notify the peer via HAL.	
Available via	ipcs-shm.h	

4.2.1.6 ipcsShmUnmanagedAcquire(P1-OP06)

Field	Content	
Service Name	ipcsShmUnmanagedAcquire	
Syntax	void *ipcsShmUnmanagedAcquire(const uint8_t instance, uint8_t chan_id)	
Service ID [hex]	N/A	
Sync/Async	Synchronous	
Reentrancy	Conditionally Reentrant Reentrancy is not supported for the same channel, but is supported for different channels/different instances	
Parameters (in)	Instance	The index of instance
	chan_id	The identify of specified channel
Parameters (inout)	None	None
Parameters (out)	None	None
Return value	void *	The pointer of payload buffer
Description	Acquire the local shared area of the non-managed channel. It is advised to obtain it only once after initialization.	
Available via	ipcs-shm.h	

4.2.1.7 ipcsShmUnmanagedTx(P1-OP07)

Field	Content	
Service Name	ipcsShmUnmanagedTx	
Syntax	int8_t ipcsShmUnmanagedTx(const uint8_t instance, uint8_t chan_id)	
Service ID [hex]	N/A	
Sync/Async	Synchronous	
Reentrancy	Conditionally Reentrant	

	Reentrancy is not supported for the same channel, but is supported for different channels/different instances	
Parameters (in)	Instance	The index of instance
	chan_id	The identify of specified channel
Parameters (inout)	None	None
Parameters (out)	None	None
Return value	int8_t	Error code
Description	Trigger a notification once the application has completed writing, allowing the peer to detect the presence of new data.	
Available via	ipcs-shm.h	

4.2.1.8 ipcsShmIsRemoteReady(P1-OP08)

Field	Content	
Service Name	ipcsShmIsRemoteReady	
Syntax	int8_t ipcsShmIsRemoteReady(const uint8_t instance)	
Service ID [hex]	N/A	
Sync/Async	Synchronous	
Reentrancy	Conditionally Reentrant Reentrancy is not supported for the same instance, but is supported for different instances	
Parameters (in)	None	None
Parameters (inout)	None	None
Parameters (out)	None	None
Return value	int8_t	Error code
Description	It is advised to invoke this prior to the initial send operation, in order to avoid transmission failures or unexpected timing issues resulting from the peer side not being ready.	
Available via	ipcs-shm.h	

4.2.1.9 ipcsShmPollChannels(P1-OP09)

Field	Content	
--------------	---------	--

Service Name	ipcsShmPollChannels	
Syntax	int8_t ipcsShmPollChannels(const uint8_t instance)	
Service ID [hex]	N/A	
Sync/Async	Synchronous	
Reentrancy	Different instances support reentrancy The same instance does not support	
Parameters (in)	None	None
Parameters (inout)	None	None
Parameters (out)	None	None
Return value	int8_t	Error code
Description	In the absence of interrupt support, poll all channels in a fair manner and process them accordingly. This polling algorithm shares the core receiving logic with the interrupt-driven path.	
Available via	ipcs-shm.h	

4.2.2 外部回调接口 IF_APPSVC (端口 P1)

编号	场景	参数 (架构)	返回值 (架构)	约束与追溯
CB-01	managed 接收	数据指针、长度、通道与实例标识 (SWE.3 固化原型)	由实现约定 (如处理条数/状态)	IPCS_018; IPCS_022
CB-02	unmanaged 接收	通道内存入口、通道与实例标识	同上	IPCS_021; IPCS_022

4.2.3 内部接口 IF_OSABST (端口 P4)

编号	架构操作	目的	输入	输出 / 失败语义	适用性
P4-OP01	ipcsOsInit	建立实例 OS 上下文: SHM 视图、接收通知路径、与 Drv_Ipcs_Hal_Cmp 衔接	instance、cfg、rxCb	成功/错误	全变体
P4-OP02	ipcsOsFree	释放 OS 资源, 与 OP01 对称	instance	void 或错误	全变体
P4-OP03	ipcsOsGetLocalShm	查询本地 SHM 基址	instance	uintptr_t	全变体

P4-OP04	ipcsOsGetRemoteShm	查询对端 SHM 本端可见基址	instance	uintptr_t	全变体
P4-OP05	ipcsOsPollChannels	轮询桥接，推进接收	instance	计数或错误	全变体
P4-OP06	ipcsShmHardIrq	核间中断服务函数，实现通知 OS 线程实现 message 的延迟处理	void	void	RTOS

4.2.4 内部接口 IF_HWABST (端口 P5)

编号	架构操作	目的	输入	输出 / 失败语义	适用性
P5-OP01	ipcsHwInit	初始化实例核间通知硬件上下文	instance、cfg	成功/错误	全变体
P5-OP02	ipcsHwFree	释放硬件上下文	instance	void	全变体
P5-OP03	ipcsHwGetRxIrq	解析 Linux 接收 IRQ 号	instance	IRQ 或负错误	仅 Linux
P5-OP04	ipcsHwIrqEnable	使能接收通知	instance	void	全变体
P5-OP05	ipcsHwIrqDisable	关闭接收通知	instance	void	全变体
P5-OP06	ipcsHwIrqNotify	发送路径触发对端通知	instance	void	全变体
P5-OP07	ipcsHwIrqClear	清除接收挂起	instance	void	全变体
P5-OP08	ipcsHwFlushCacheLocal	Flush 本地 cache	instance	void	全变体
P5-OP09	ipcsHwFlushCacheRemote	Flush 远端 cache	instance	void	全变体

4.3 FUNCTIONAL DESCRIPTION 功能描述

IPCS Driver 提供以下功能：

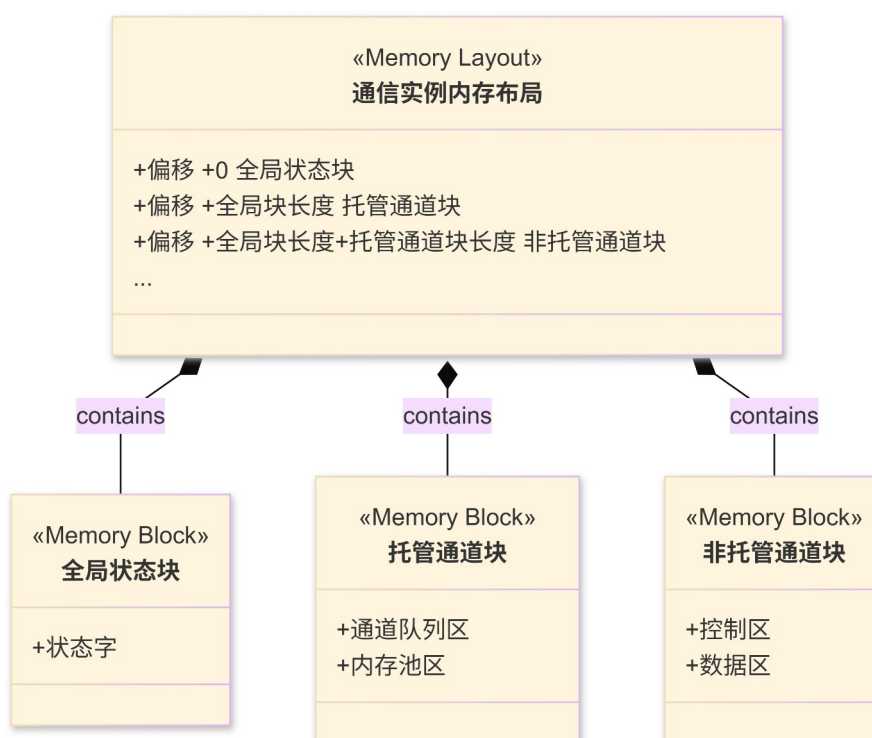
- 系统初始化——初始化 IPC 实例及资源
- 资源释放——释放所有 IPC 实例
- Buffer 管理——分配、释放共享内存 buffer
- 数据发送——发送数据到远端
- 数据接收——接收并处理远端数据
- 通道管理——管理托管通道、非托管通道
- SOC 核间通知——通过核间中断实现
- 轮询处理——支持非中断模式接收、处理数据

4.4 DATA ARCHITECTURE DESIGN 数据架构设计

根据 IPCS_014、IPCS_015、IPCS_016、IPCS_017、IPCS_018 等需求项中对共享内存的相关描述，以及组件设计中对共享内存架构的强依赖性，需要对共享内存进行具体的设计以完成对需求的响应和对组件设计的支撑。IPCS 的核心操作对象是两端对称映射的共享区在 Local 窗口与 Remote 窗口中的视图。Drv_Ipcs_Osal_Cmp 提供 ipcsOsGetLocalShm / ipcsOsGetRemoteShm；Drv_Ipcs_Core_Cmp 按与对端相同规则计算通道起点；在已划定偏移上建立双环与 BD 等通道、内存池的初始化创建与管理都涉及具体的共享内存布局设计。注意，后续章节的内存布局图中，只展示了单测共享内存区域的内存布局，对端采用相同的布局设计。

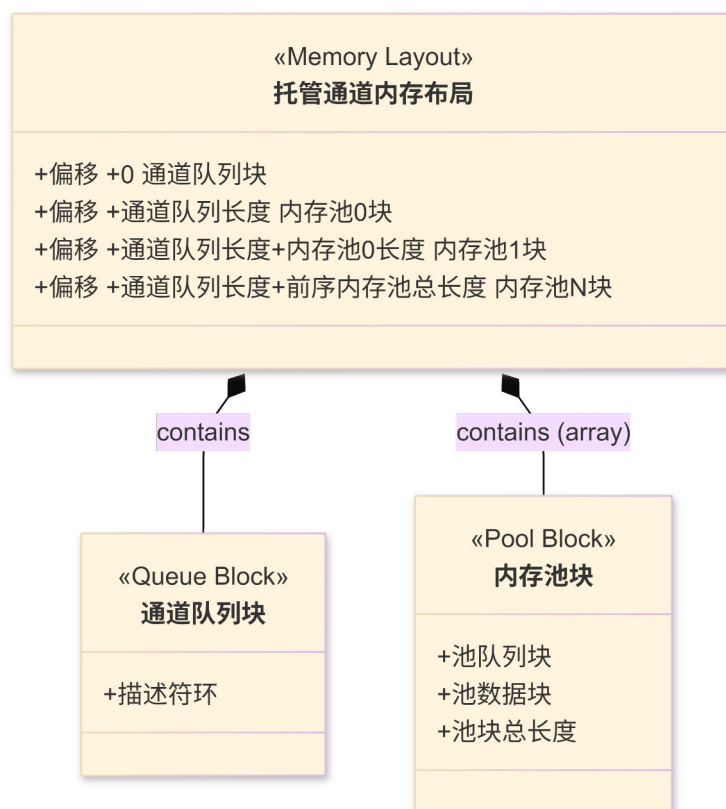
4.4.1 INSTANCE MEMORY LAYOUT 通信实例级内存布局

同一 instance 在 Local 与 Remote 两侧各有一块等长逻辑区（配置对称）；通道在配置数组中的顺序决定条带拼接顺序，与类型无关。按照通道配置顺序依次排布。如下图所示，单个通信实例级内存布局起始地址为全局初始化标识，后续为一次排列的通道条带；这些内存布局对象为组件接口中的对端初始化状态读取、通道处理等提供了对象支撑。



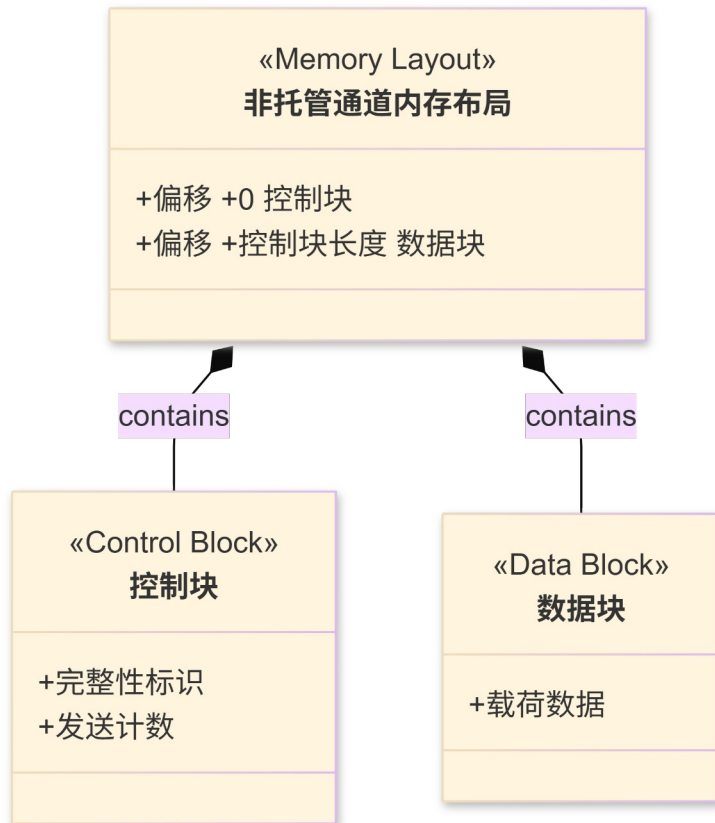
4.4.2 MANAGED CHANNEL 内存布局

在通道实例内存布局中，Managed Channel 内存内部设计如下图所示。起始位置为当前通道的收发管理队列，本端在发送数据时候执行 push 操作，对端在接收时候执行 pop 操作；后续依次排列为可配置的 buffer pool 内存带，buffer pool 是一组根据配置参数（个数、大小）设置的内存结构，其结构为：buffer pool 管理队列结构作为 header，实际的内存空间作为本端的 buffer。



4.4.3 UNMANAGED CHANNEL 内存布局

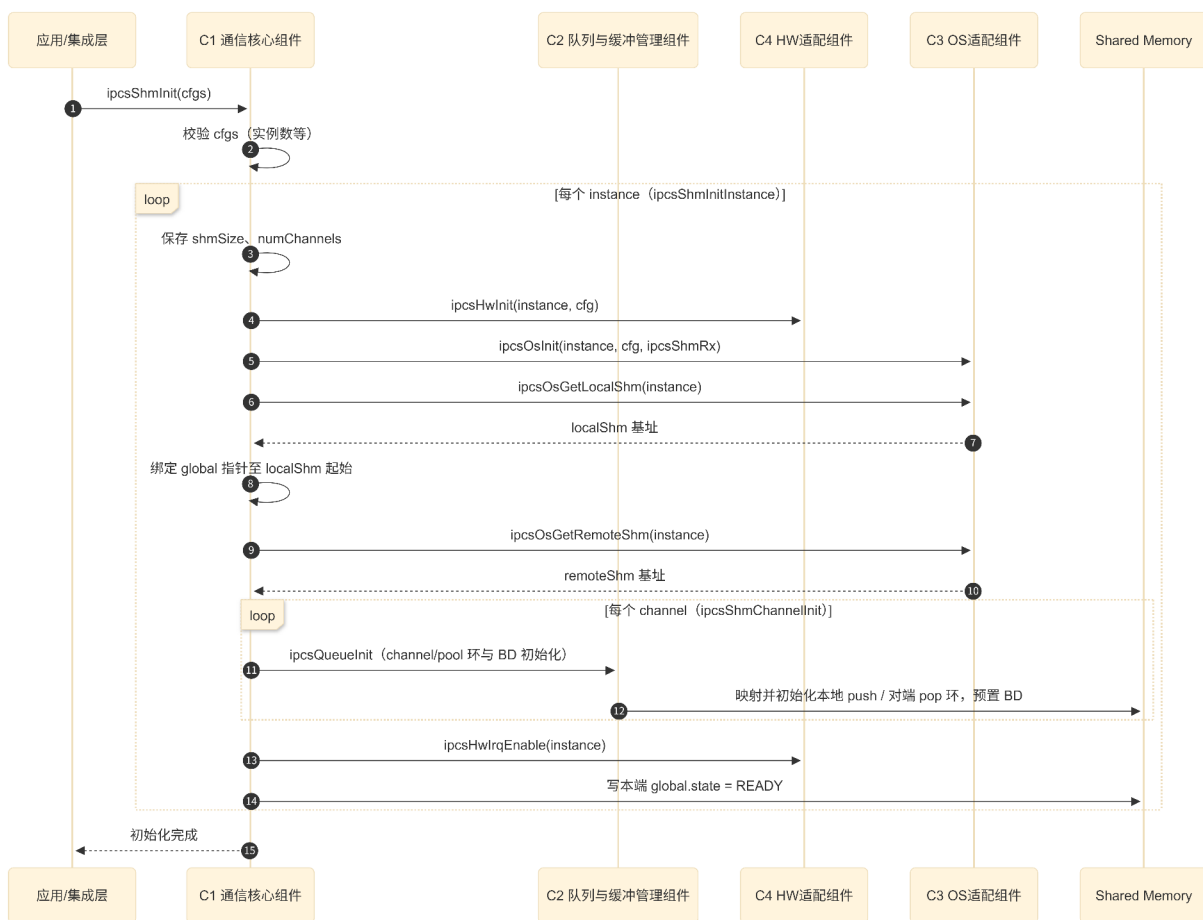
Unmanaged Channel 是通道实例级内存布局上的一个 Remote、Local 配置对称的内存空间。其内存起始位置为一个 32bits 的哨兵元素用于指示内存可用性，后续为 32bits 的发送计数值以及实际的 buffer 空间。



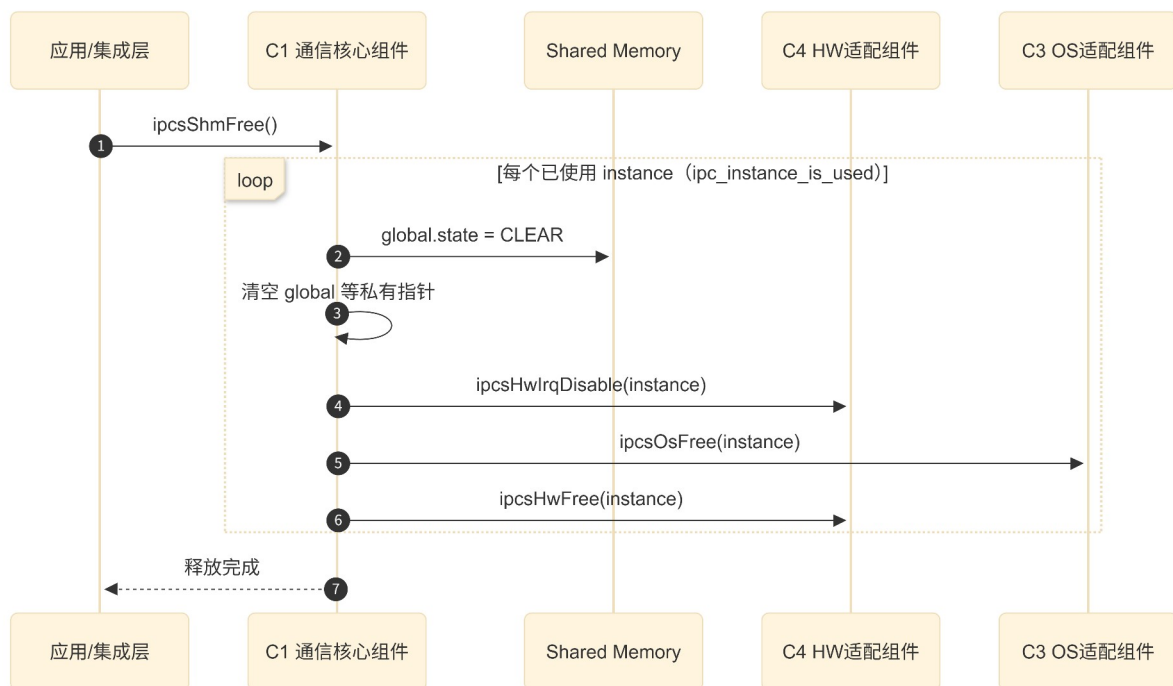
4.5 DYNAMIC ARCHITECTURE 动态架构图

4.5.1 USE CASE SEQUENCE DIAGRAM 用例序列图

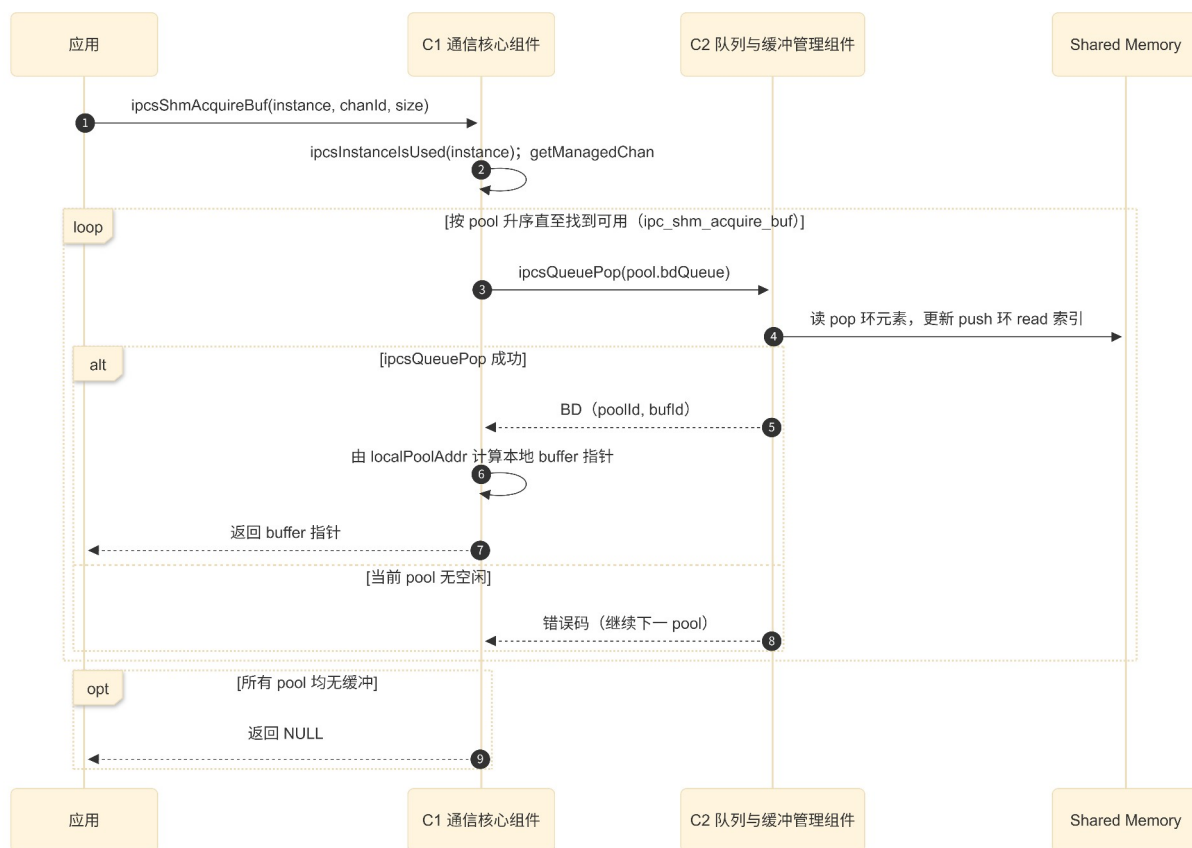
4.5.1.1 IPCS 初始化接口时序



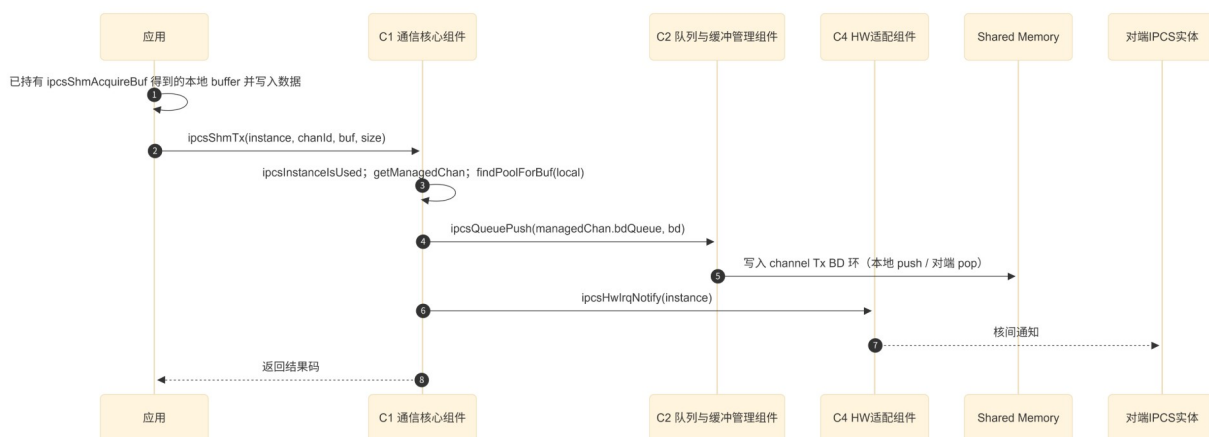
4.5.1.2 IPCS 释放接口时序



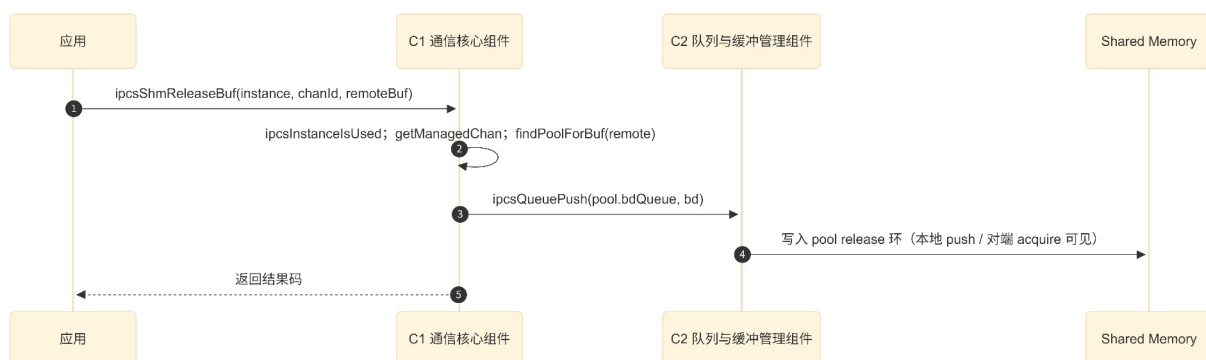
4.5.1.3 IPCS managed buffer 获取接口时序



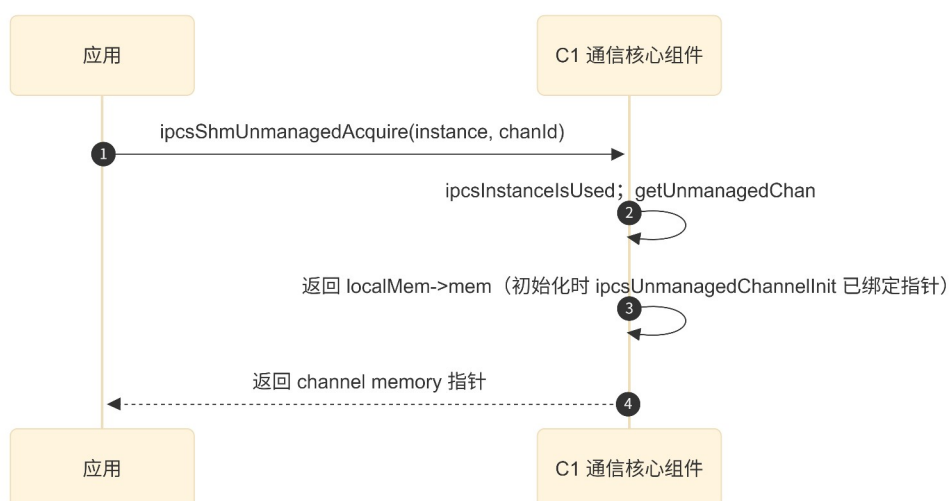
4.5.1.4 IPCS managed 发送接口时序



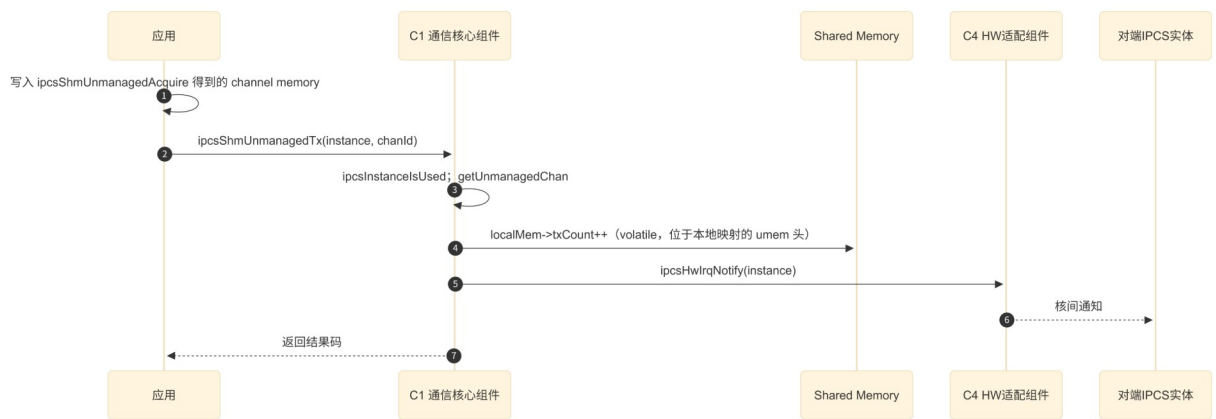
4.5.1.5 IPCS managed buffer 释放接口时序



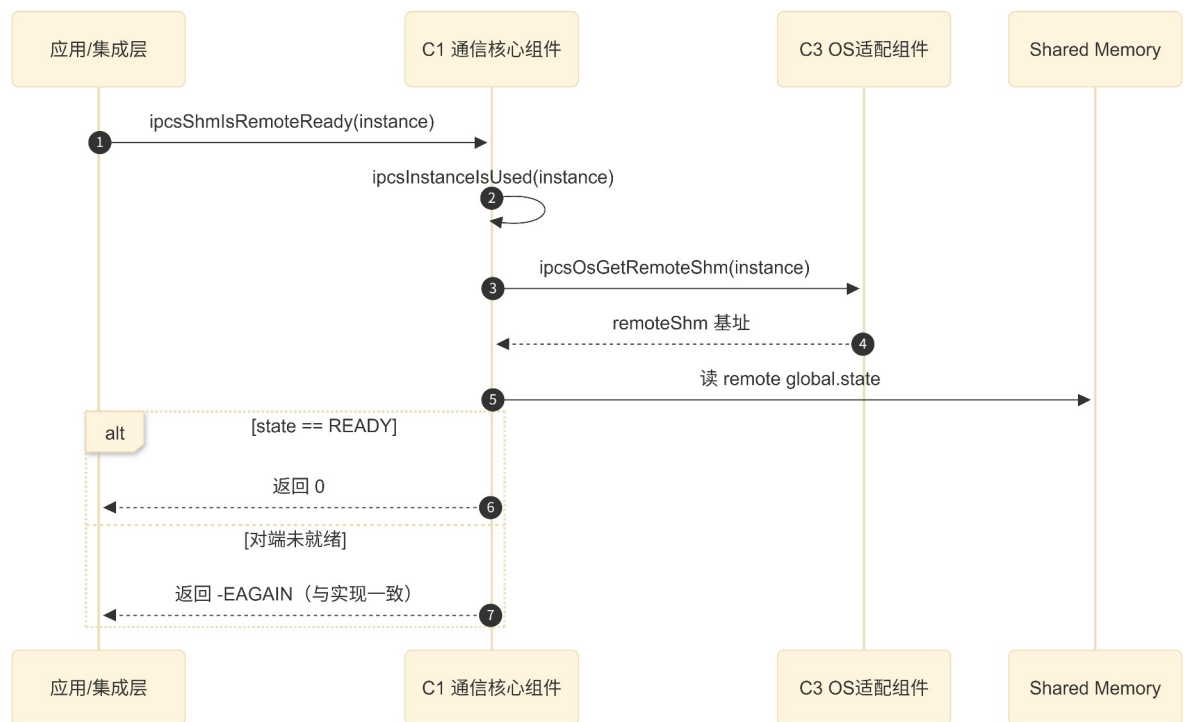
4.5.1.6 IPCS unmanaged memory 获取接口时序



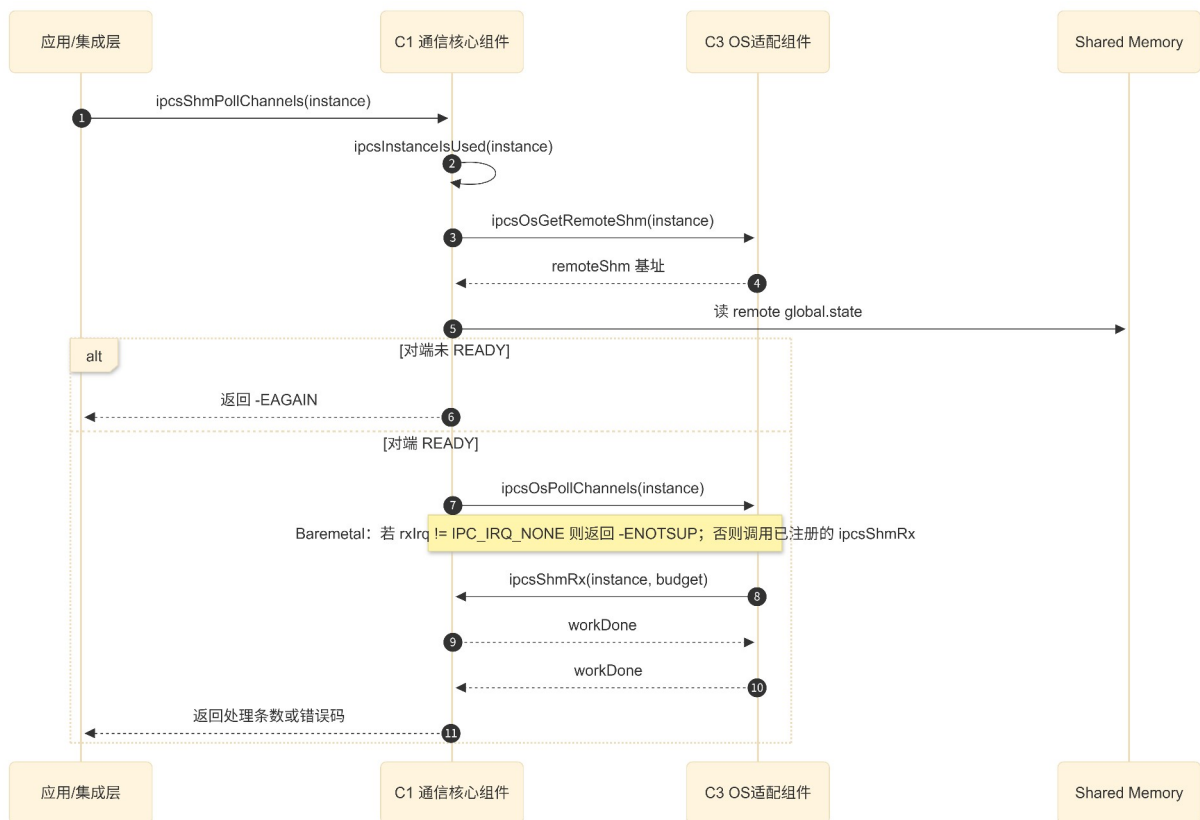
4.5.1.7 IPCS unmanaged 发送通知接口时序



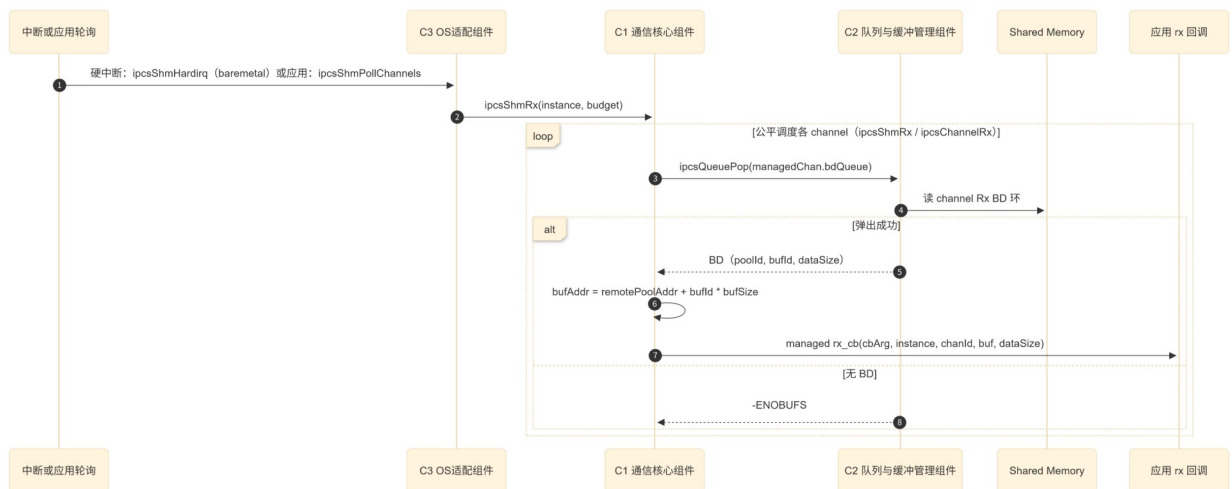
4.5.1.8 IPCS 对端 ready 检查接口时序



4.5.1.9 IPCS Polling 接收时序图



4.5.1.10 IPCS managed channel 接收分发内部时序



4.5.1.11 IPCS unmanaged channel 接收分发内部时序

